

■ Bus Ticket Booking System

A Complete Step-by-Step Java Project Guide for Students

Console-Based | Object-Oriented Java | Beginner–Intermediate

Java 17+

5 Classes

OOP Concepts

~400 Lines

■ This guide walks students through building a fully functional console-based Bus Ticket Booking System in Java using the exact source code provided. Every class, field, and method is explained in detail so students understand not just what to write, but why.

■ Table of Contents

- 1 Project Overview & Learning Objectives
- 2 Prerequisites & Project Setup
- 3 Project Structure & Package
- 4 Step 1 — Customer.java
- 5 Step 2 — Bus.java
- 6 Step 3 — Ticket.java
- 7 Step 4 — BookingSystem.java (Skeleton + Constructor)

- 8 Step 5 — `signUp()` and `login()`
- 9 Step 6 — `showAvailableBuses()` and `bookTicket()`
- 10 Step 7 — `viewTickets()` and `cancelTicket()`
- 11 Step 8 — `showBusSummary()`
- 12 Step 9 — `Main.java` (Entry Point)
- 13 How to Compile & Run
- 14 Sample Test Walkthrough
- 15 Common Errors & Fixes
- 16 Extension Challenges

1. Project Overview & Learning Objectives

The Bus Ticket Booking System is a menu-driven Java console application. It is split across five source files inside the package **com.busticketbooking**. Customers can sign up, log in, view buses, book seats, view or cancel tickets, and see bus summaries.

Concept	Where Used
Classes & Objects	Customer, Bus, Ticket, BookingSystem, Main
Encapsulation	All private fields accessed via public getters
ArrayList & List	customers, buses, tickets in BookingSystem
boolean[] array	seats[] in Bus — true=booked, false=available
Scanner input	All interactive methods (signUp, login, bookTicket...)
Loops & switch-case	Menu loop in Main, for-each in all search methods
String .equals()	Password matching in login()
0-based vs 1-based index	seatNumber - 1 conversion in bookTicket()

2. Prerequisites & Project Setup

- JDK 17 or higher installed (java -version to check)
- IDE recommended: IntelliJ IDEA, Eclipse, or VS Code with Java Extension Pack
- Basic Java knowledge: variables, loops, methods, classes

Create the project in IntelliJ IDEA:

- File → New → Project → Java (no frameworks needed)
- Name the project: BusTicketBookingSystem
- Inside src/, right-click → New → Package → type: com.busticketbooking
- Inside that package, create 5 Java classes: Customer, Bus, Ticket, BookingSystem, Main

■■ Every file must start with: `package com.busticketbooking;` as the first line.

3. Project Structure & Package

```
BusTicketBookingSystem/  
  ■■■ src/  
    ■■■ com/  
      ■■■ busticketbooking/  
        ■■■ Customer.java  
        ■■■ Bus.java  
        ■■■ Ticket.java  
        ■■■ BookingSystem.java  
        ■■■ Main.java
```

- All five files are in the same package (`com.busticketbooking`), so they can reference each other directly without any import statements between them.

Step 1 Customer.java — Stores Customer Details

The **Customer** class is the simplest class in the project. It stores the personal details of a registered user. All fields are **private** (encapsulation) and exposed only through public getter methods. There are no setter methods because customer data does not change after registration.

Fields explained:

Field	Type	Purpose
customerId	int	Unique ID auto-assigned at registration
name	String	Full name entered during sign-up
password	String	Login password (plain text for simplicity)
age	int	Customer's age
gender	String	Male / Female / Other

■ Customer.java package com.busticketbooking

```
package com.busticketbooking;

public class Customer {
    private int customerId;
    private String name;
    private String password;
    private int age;
    private String gender;

    public Customer(int customerId, String name, String password,
        int age, String gender) {
        super();
        this.customerId = customerId;
        this.name = name;
        this.password = password;
        this.age = age;
        this.gender = gender;
    }

    public int getCustomerId() { return customerId; }
    public String getName() { return name; }
    public String getPassword() { return password; }
    public int getAge() { return age; }
    public String getGender() { return gender; }
}
```

■ There is no toString() in Customer — the booking system prints customer info directly using the getters.

Step 2 Bus.java — Manages Bus Seats

The **Bus** class models a real bus. The most important field is **seats[]** — a boolean array where each index represents one seat. **false** = available, **true** = booked. The array is sized to *totalSeats* in the constructor and starts fully available (Java initialises boolean arrays to false automatically).

Key methods explained:

- **getAvailableSeats()** — Loops through `seats[]`, counts every false entry.
- **showAvailableSeats()** — Prints seat numbers (1-based) where `seats[i]` is false.
- **bookSeat(int seatNumber)** — Receives a 0-based index. Marks `seats[seatNumber]=true`. Returns false if already booked.
- **cancelSeat(int seatNumber)** — Receives a 0-based index. Sets `seats[seatNumber]=false`. Returns false if not booked.

■ Bus.java package com.busticketbooking

```
package com.busticketbooking;

import java.util.Arrays;

public class Bus {
    private int busId;
    private String busType; // AC / Non-AC
    private String seatType; // Seater / Sleeper
    private int totalSeats;
    private boolean[] seats; // true=booked, false=available

    public Bus(int busId, String busType, String seatType, int totalSeats) {
        super();
        this.busId = busId;
        this.busType = busType;
        this.seatType = seatType;
        this.totalSeats = totalSeats;
        this.seats = new boolean[totalSeats]; // all false by default
    }

    public int getAvailableSeats() {
        int count = 0;
        for (boolean seat : seats) {
            if (seat == false) { count++; }
        }
        return count;
    }

    public void showAvailableSeats() {
        System.out.println("Available seats for Bus " + busId + ":");
        for (int i = 0; i < totalSeats; i++) {
            if (!seats[i]) {
                System.out.print((i + 1) + " "); // print 1-based seat number
            }
        }
        System.out.println();
    }

    // seatNumber is 0-based (converted in BookingSystem)
    public boolean bookSeat(int seatNumber) {
```

```

if (seatNumber < 0 || seatNumber >= totalSeats) return false;
if (seats[seatNumber] == false) {
    seats[seatNumber] = true;
    return true;
}
return false; // already booked
}

public boolean cancelSeat(int seatNumber) {
    if (seatNumber < 0 || seatNumber >= totalSeats) return false;
    if (seats[seatNumber] == true) {
        seats[seatNumber] = false;
        return true;
    }
    return false; // seat was not booked
}

public int getBusId() { return busId; }
public String getBusType() { return busType; }
public String getSeatType() { return seatType; }
public int getTotalSeats(){ return totalSeats; }

@Override
public String toString() {
    return "Bus [busId=" + busId + ", busType=" + busType +
        ", seatType=" + seatType + ", totalSeats=" + totalSeats +
        ", seats=" + Arrays.toString(seats) + "]\n";
}
}

```

■ ■ Important: `bookSeat()` and `cancelSeat()` both receive a 0-based index. The conversion from the user's 1-based input (`seatNumber - 1`) happens in `BookingSystem.bookTicket()`.

Step 3 Ticket.java — Records a Booking

A **Ticket** links a customer to a bus. It stores the list of booked seat indices (0-based), the total fare, and how many seats were booked. Seats are added one at a time using **addBookedSeat()** after each successful **bookSeat()** call in **BookingSystem**.

■ **Ticket.java** package `com.busticketbooking`

```
package com.busticketbooking;

import java.util.ArrayList;
import java.util.List;

public class Ticket {
    private int ticketId;
    private int customerId;
    private Bus bus;
    private int numberOfTickets;
    private double fare;
    private List<Integer> bookedSeats; // stores 0-based seat indices

    public Ticket(int ticketId, Bus bus, int numberOfTickets,
        double fare, int customerId) {
        super();
        this.ticketId = ticketId;
        this.bus = bus;
        this.numberOfTickets = numberOfTickets;
        this.fare = fare;
        this.customerId = customerId;
        this.bookedSeats = new ArrayList<>();
    }

    // Called after each successful bookSeat()
    public void addBookedSeat(int seatNumber) {
        bookedSeats.add(seatNumber);
    }

    public int getTicketId() { return ticketId; }
    public List<Integer> getBookedSeats() { return bookedSeats; }
    public Bus getBus() { return bus; }
    public int getNumberOfTickets() { return numberOfTickets; }
    public double getFare() { return fare; }
    public int getCustomerId() { return customerId; }

    @Override
    public String toString() {
        return "Ticket [ticketId=" + ticketId + ", customerId=" + customerId +
            ", bus=" + bus + ", numberOfTickets=" + numberOfTickets +
            ", fare=" + fare + ", bookedSeats=" + bookedSeats + "]\n";
    }
}
```

■ **bookedSeats** stores 0-based indices (matching the `seats[]` array in **Bus**). When displaying to the user, add 1: seat + 1 to show the human-readable seat number.

Step 4 BookingSystem.java — Skeleton & Constructor

BookingSystem is the controller class. It owns three ArrayLists (customers, buses, tickets), three ID counters, and a reference to the currently logged-in customer. The constructor calls **initializeBuses()** to pre-load four sample buses so the application has data ready from startup.

■ BookingSystem.java — fields, constructor & initializeBuses() package com.busticketbooking

```
package com.busticketbooking;

import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class BookingSystem {

    private List<Customer> customers;
    private List<Bus> buses;
    private List<Ticket> tickets;
    private int customerIdCounter; // starts at 1, increments on each sign-up
    private int busIdCounter; // starts at 1, increments in initializeBuses()
    private int ticketIdCounter; // starts at 1, increments on each booking
    private Customer currentCustomer; // null when nobody is logged in

    public BookingSystem() {
        customers = new ArrayList<>();
        buses = new ArrayList<>();
        tickets = new ArrayList<>();
        customerIdCounter = 1;
        busIdCounter = 1;
        ticketIdCounter = 1;
        currentCustomer = null;
        initializeBuses(); // pre-load sample data
    }

    public void initializeBuses() {
        // busIdCounter++ uses current value THEN increments
        buses.add(new Bus(busIdCounter++, "AC", "Seater", 30));
        buses.add(new Bus(busIdCounter++, "Non-AC", "Seater", 40));
        buses.add(new Bus(busIdCounter++, "AC", "Sleeper", 20));
        buses.add(new Bus(busIdCounter++, "Non-AC", "Sleeper", 25));
    }

    // ... methods added in the following steps
}
```

■ `busIdCounter++` is the post-increment operator: the current value is used as the Bus ID first, then the counter goes up. Bus 1 gets ID 1, Bus 2 gets ID 2, and so on.

Step 5 BookingSystem.java — signUp() and login()

signUp() collects five pieces of information, creates a Customer object with the next available ID, and adds it to the customers list. **login()** loops through all customers comparing ID and password — it uses **.equals()** (not **==**) to compare the password String.

■ BookingSystem.java — signUp() package com.busticketbooking

```
public void signUp(Scanner scanner) {
    System.out.println("Enter your name:");
    String name = scanner.nextLine();

    System.out.println("Enter your password:");
    String password = scanner.nextLine();

    System.out.println("Enter your age:");
    int age = scanner.nextInt();
    scanner.nextLine(); // consume leftover newline after nextInt()

    System.out.println("Enter your gender:");
    String gender = scanner.nextLine();

    Customer customer = new Customer(
        customerIdCounter++, name, password, age, gender);
    customers.add(customer);
    System.out.println(
        "Sign up successful! Your customer ID is: " + customer.getCustomerId());
}
```

■ BookingSystem.java — login() package com.busticketbooking

```
public boolean login(Scanner scanner) {
    System.out.println("Enter your customer ID:");
    int id = scanner.nextInt();
    scanner.nextLine(); // consume leftover newline

    System.out.println("Enter your password:");
    String password = scanner.nextLine();

    for (Customer customer : customers) {
        if (customer.getCustomerId() == id
            && customer.getPassword().equals(password)) {
            currentCustomer = customer;
            System.out.println(
                "Login successful! Welcome, " + customer.getName());
            return true;
        }
    }
    System.out.println("Invalid credentials!");
    return false;
}
```

■■ scanner.nextLine() after scanner.nextInt() is essential. nextInt() reads the number but leaves the newline character in the buffer. The following nextLine() would then read that empty newline instead of the user's actual input.

Step 6 BookingSystem.java — showAvailableBuses() and bookTicket()

■ BookingSystem.java — showAvailableBuses() package com.busticketbooking

```
public void showAvailableBuses() {
    System.out.println("\nAvailable Buses:");
    for (Bus bus : buses) {
        System.out.println(bus); // calls Bus.toString()
    }
}
```

bookTicket() is the most complex method. Read through each numbered comment to understand the flow:

■ BookingSystem.java — bookTicket() package com.busticketbooking

```
public void bookTicket(Scanner scanner) {
    // 1. Guard: must be logged in
    if (currentCustomer == null) {
        System.out.println("Please login first!");
        return;
    }

    showAvailableBuses();

    // 2. Select a bus by ID
    System.out.println("\nEnter bus ID:");
    int busId = scanner.nextInt();
    scanner.nextLine();

    Bus selectedBus = null;
    for (Bus bus : buses) {
        if (bus.getBusId() == busId) { selectedBus = bus; break; }
    }
    if (selectedBus == null) {
        System.out.println("Invalid bus ID!"); return;
    }

    // 3. Show seat map and ask how many tickets
    selectedBus.showAvailableSeats();
    System.out.println("Enter number of tickets to book:");
    int numberOfTickets = scanner.nextInt();
    scanner.nextLine();

    if (numberOfTickets > selectedBus.getAvailableSeats()) {
        System.out.println("Not enough seats available!"); return;
    }

    // 4. Collect seat choices one by one
    List<Integer> seatsToBook = new ArrayList<>();
    for (int i = 0; i < numberOfTickets; i++) {
        System.out.println("Enter seat number for ticket " + (i+1) + ":");
        int seatNumber = scanner.nextInt() - 1; // convert 1-based -> 0-based
        scanner.nextLine();

        if (seatNumber < 0 || seatNumber >= selectedBus.getTotalSeats()) {
            System.out.println("Invalid seat number!"); return;
        }
        if (!selectedBus.bookSeat(seatNumber)) {
            System.out.println("Seat " + (seatNumber+1) + " is already booked!");
            return;
        }
    }
}
```

```

seatsToBook.add(seatNumber);
}

// 5. Calculate fare: AC=1000 base, Sleeper adds 500
double baseFare = selectedBus.getBusType().equals("AC") ? 1000 : 500;
baseFare += selectedBus.getSeatType().equals("Sleeper") ? 500 : 0;
double totalFare = baseFare * numberOfTickets;

// 6. Create ticket and register booked seats
Ticket ticket = new Ticket(ticketIdCounter++, selectedBus,
numberOfTickets, totalFare,
currentCustomer.getCustomerId());
for (int seat : seatsToBook) { ticket.addBookedSeat(seat); }
tickets.add(ticket);

System.out.println("\nBooking successful!");
System.out.println("Ticket ID: " + ticket.getTicketId());
System.out.println("Total Fare: " + totalFare);
System.out.println("Booked Seats: " + seatsToBook);
}

```

■ Fare logic: AC Seater=Rs.1000 | Non-AC Seater=Rs.500 | AC Sleeper=Rs.1500 | Non-AC Sleeper=Rs.1000. The ternary operators handle this in two lines.

Step 7 BookingSystem.java — viewTickets() and cancelTicket()

■ BookingSystem.java — viewTickets() package com.busticketbooking

```
public void viewTickets() {
    if (currentCustomer == null) {
        System.out.println("Please login first!"); return;
    }
    System.out.println("\nYour Tickets:");
    boolean found = false;
    for (Ticket ticket : tickets) {
        if (ticket.getCustomerId() == currentCustomer.getCustomerId()) {
            System.out.println(ticket); // calls Ticket.toString()
            found = true;
        }
    }
    if (!found) System.out.println("No tickets found!");
}
```

■ BookingSystem.java — cancelTicket() package com.busticketbooking

```
public void cancelTicket(Scanner scanner) {
    if (currentCustomer == null) {
        System.out.println("Please login first!"); return;
    }

    viewTickets(); // show current tickets first
    System.out.println("\nEnter ticket ID to cancel:");
    int ticketId = scanner.nextInt();
    scanner.nextLine();

    // Find the ticket belonging to the current customer
    Ticket ticketToCancel = null;
    for (Ticket ticket : tickets) {
        if (ticket.getTicketId() == ticketId
            && ticket.getCustomerId() == currentCustomer.getCustomerId()) {
            ticketToCancel = ticket;
            break;
        }
    }
    if (ticketToCancel == null) {
        System.out.println("Invalid ticket ID!"); return;
    }

    // Free every seat that was booked on this ticket
    Bus bus = ticketToCancel.getBus();
    for (int seat : ticketToCancel.getBookedSeats()) {
        bus.cancelSeat(seat); // seat is 0-based index
    }

    tickets.remove(ticketToCancel);
    System.out.println("Ticket cancelled successfully!");
    System.out.println("Refund amount: " + ticketToCancel.getFare());
}
```

■ `cancelTicket()` checks both the ticket ID AND the customer ID. This prevents one customer from cancelling another customer's ticket.

Step 8 BookingSystem.java — showBusSummary()

`showBusSummary()` gives an overview of every bus — total seats, how many are booked, and how many remain available. It calculates booked seats as *totalSeats* - *availableSeats*.

■ BookingSystem.java — showBusSummary() package com.busticketbooking

```
public void showBusSummary() {
    System.out.println("\nBus Summary:");
    for (Bus bus : buses) {
        System.out.println("Bus ID: " + bus.getBusId());
        System.out.println("Type: " + bus.getBusType());
        System.out.println("Seat Type: " + bus.getSeatType());
        System.out.println("Total Seats: " + bus.getTotalSeats());
        System.out.println("Booked Seats: " +
            (bus.getTotalSeats() - bus.getAvailableSeats()));
        System.out.println("Available Seats: " + bus.getAvailableSeats());
        System.out.println(); // blank line between buses
    }
}
```

Step 9 Main.java — Entry Point & Menu Loop

Main contains only the **main()** method. It creates one instance of `BookingSystem` and a single `Scanner`, then runs an infinite while loop that displays the menu and dispatches choices via a switch-case.

■ **Main.java** package `com.busticketbooking`

```
package com.busticketbooking;

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        BookingSystem system = new BookingSystem();
        Scanner scanner = new Scanner(System.in);

        while (true) {
            System.out.println("\nBus Ticket Booking System");
            System.out.println("1. Sign Up");
            System.out.println("2. Login");
            System.out.println("3. Show Available Buses");
            System.out.println("4. Book Ticket");
            System.out.println("5. View Tickets");
            System.out.println("6. Cancel Ticket");
            System.out.println("7. Show Bus Summary");
            System.out.println("8. Exit");
            System.out.println("Enter your choice:");

            int choice = scanner.nextInt();
            scanner.nextLine(); // consume newline after menu choice

            switch (choice) {
                case 1: system.signUp(scanner); break;
                case 2: system.login(scanner); break;
                case 3: system.showAvailableBuses(); break;
                case 4: system.bookTicket(scanner); break;
                case 5: system.viewTickets(); break;
                case 6: system.cancelTicket(scanner); break;
                case 7: system.showBusSummary(); break;
                case 8:
                    System.out.println(
                        "Thank you for using Bus Ticket Booking System!");
                    scanner.close();
                    return; // exits main(), ends the program
                default:
                    System.out.println("Invalid choice!");
            }
        }
    }
}
```

■ `scanner.close()` is called only on exit (case 8). Never close the scanner inside a method that is called repeatedly — closing it makes `System.in` unreadable for the rest of the program's life.

13. How to Compile & Run

From the command line (inside the project root):

```
# Compile all files in the package directory
javac src/com/busticketbooking/*.java

# Run using the fully-qualified class name
java -cp src com.busticketbooking.Main
```

■ In IntelliJ: just press the green Run button on Main.java — the IDE handles compilation automatically.

14. Sample Test Walkthrough

Step	Menu Choice	Input	Expected Output
1	1 – Sign Up	Alice / pass123 / 22 / Female	Sign up successful! Your customer ID is: 1
2	2 – Login	ID: 1 / pass123	Login successful! Welcome, Alice
3	3 – Show Buses	(none)	List of 4 buses with seat details
4	4 – Book Ticket	Bus ID: 1 / 2 tickets / seats 3 and 7	Booking successful! Ticket ID: 1 Fare: 2000.0
5	5 – View Tickets	(none)	Ticket details printed including booked seats [2, 6]
6	6 – Cancel Ticket	Ticket ID: 1	Ticket cancelled! Refund: 2000.0
7	7 – Bus Summary	(none)	All buses with booked/available counts
8	8 – Exit	(none)	Thank you for using Bus Ticket Booking System!

■ ■ Note on seat numbers: The user types 1-based seat numbers (e.g. 3 and 7). `bookTicket()` subtracts 1 before calling `bookSeat()`, so `seats[2]` and `seats[6]` are marked true. `viewTickets()` prints [2, 6] (0-based) — this is expected behaviour.

15. Common Errors & Fixes

■ *InputMismatchException*

Cause: Calling `scanner.nextInt()` then `scanner.nextLine()` without consuming the newline.

Fix: Always add `scanner.nextLine()` immediately after every `nextInt()` or `nextDouble()` call.

■ *Login always fails (== vs .equals())*

Cause: Using `==` to compare password strings compares memory addresses, not content.

Fix: Always use `password.equals(input)` for String comparisons. Already done correctly in `login()`.

■ *ArrayIndexOutOfBoundsException in bookSeat()*

Cause: User enters a seat number of 0 or larger than `totalSeats`.

Fix: `Bus.bookSeat()` guards with: `if (seatNumber < 0 || seatNumber >= totalSeats) return false;`

■ *Seat shows as available after cancellation*

Cause: `cancelSeat()` not called, or called with the wrong index.

Fix: `cancelTicket()` loops through `bookedSeats` (0-based) and calls `bus.cancelSeat(seat)` for each.

■ *NullPointerException on currentCustomer*

Cause: Calling `bookTicket()` / `viewTickets()` / `cancelTicket()` before logging in.

Fix: Each method checks: `if (currentCustomer == null) { print error; return; }` at the top.

■ *Wrong seat numbers displayed in viewTickets()*

Cause: `bookedSeats` stores 0-based indices but user expects 1-based numbers.

Fix: Expected behaviour — the list `[2, 6]` means the user booked seats 3 and 7. Optionally fix `toString()` in `Ticket` to add 1.

16. Extension Challenges

Try these after finishing the base project

Level	Challenge
■ Easy	Add a logout option (set <code>currentCustomer = null</code>) to the menu.
■ Easy	Display available seat count next to each bus in <code>showAvailableBuses()</code> .
■ Easy	Add senior citizen discount: customers older than 60 pay 20% less.
■■ Medium	Allow the admin to add a new bus at runtime through a menu option.
■■ Medium	Display booked seats as 1-based numbers in <code>viewTickets()</code> by adding 1.
■■ Medium	Add file I/O: save all data to a <code>.txt</code> file and reload it on startup.
■■■ Hard	Replace <code>ArrayList</code> with <code>HashMap</code> keyed by ID.
■■■ Hard	Add an admin role with a hardcoded password that can view revenue per bus.

■ You have now built a complete, working Bus Ticket Booking System in Java across 5 classes. Review the UML diagram to confirm every field, method, and relationship has been implemented correctly.